

# 数据结构

更多咨询：北京信息学窦老师QQ3377089232

# 1、栈和队列

# 栈的操作

栈的容器: `int s[MAXN]` or `stack<int> s`

读取顶层元素: `s[top]` or `s.top()`

将元素压入栈顶: `s[++top] = x` or `s.push(x)`

删除栈顶元素: `top--` or `s.pop()`

判断栈是否为空: `top == 0` or `s.empty()`

# 铁轨

有 $n$ 节车厢按1到 $n$ 的顺序驶入车站  
车站有一个中转站，先进后出  
判断车厢能按指定的顺序出站

分析：

只要按题目的要求模拟即可

当要求 $x$ 出栈时， $x$ 在栈内而不在栈顶，则无法满足条件

# 出栈序列统计

一个栈的容量为 $m$ ，进栈序列为 $1 \sim n$  ( $1 \leq n, m \leq 1000$ )  
问有多少种不同的出栈序列？

分析：

对于1号元素，它首先进栈，然后某个时刻出栈  
它将序列分成了独立的两个部分

$f(i, j)$  表示  $i$  个元素，栈容量为  $j$ ，有多少种可能

$f(i, j) = \sum ( f(k, j-1) * f(i-k-1, j) )$  70分

设左方有 $a$ 个元素，栈内有 $b$ 个元素，从  $(n, 0)$  转移到  $(0, 0)$

$f(a, b) = f(a, b+1) + f(a+1, b-1)$  100分

# 双栈排序

通过两个栈将一个序列升序排序 ( $n \leq 1000$ )

a入S1, b出S1, c入S2, d出S2

使得操作序列的字典序尽可能小

分析:

尽量把数字压入s1, 那么什么时候必须压入s2呢?

$a[i]$ 和 $a[j]$  不能压入同一个栈 $\Leftrightarrow$ 存在一个 $k$ ,使得 $i < j < k$ 且 $a[k] < a[i] < a[j]$

不能压入同一栈则连一条边, 二分图染色

预处理 $dp[k]$ 为 $k \dots n$ 中的最小数

# 单调栈

在栈结构的基础上，满足栈内元素单调（递增或递减）

模拟一个数列构造一个单调递增栈

进栈元素分别为3，4，2，6，4，5，2，3

给定n个正数，一个区间的权值定义为：

这个区间的所有数之和乘以这个区间内最小的数

求所有区间中权值最大的区间

枚举最小值，我们想知道它向左向右最多能延伸多少

维护一个从栈底到栈顶递增的单调栈，每个数记录往左能延伸多少

# 队列的操作

队列的容器: `int q[MAXN]` or `queue<int> q`

读取队首元素: `q[head]` or `q.front()`

读取队尾元素: `q[tail]` or `q.rear()`

队尾元素进队: `q[++tail] = x` or `q.push(x)`

队首元素出队: `head++` or `q.pop()`

判断队列是否为空: `head > tail` or `q.empty()`



# 卡片游戏

桌面上有一叠 $n$ 张牌，从上往下编号 $1\sim n$

当至少还剩两张牌时进行以下操作：

把第一张牌扔掉，把新的第一张牌放到最后

输出扔掉牌的序列，以及最后剩下的一张牌

分析：

两种操作符合队列的特点

模拟即可

# 单调队列

一个队列内部的元素具有严格单调性

既可以在队首删元素，也可以在队尾删元素

模拟一个单调递增队列，进队列元素分别为3，4，2，6，4，5，2，3

给定一个长度为 $n$ 的序列，找出一段连续的长度不超过 $M$ 的子序列，和最大

第一步，将连续和转化成前缀和之差

对于一个固定的 $s_i$ ，求 $s(i-M) \sim s(i-1)$ 中的最小值

用单调递增队列维护

# 跳房子

$n$ 个格子，格子的位置为 $x_i$ ，分数为 $s_i$ ，至少获得 $k$ 分，任意时刻都能停  
每次向右弹跳固定的 $d$ ，可以修改灵活度 $g$  ( $d-g \sim d+g$ )，求 $g$ 的最小值  
( $n \leq 500000$ ,  $d \leq 2000$ ,  $x_i \leq 10^9$ )

分析：

$dp[i]$ 表示跳到第 $i$ 个格子时获得的最大得分

枚举答案，直接dp判断 20分

二分答案，直接dp判断 50分

二分答案，dp判断时用上单调队列，100分

## 2、树状数组

# 单点修改，区间查询

给定一个序列，支持连续和查询，也支持单点修改

$\text{lowbit}(x)$  表示二进制表达式中最右边的1所对应的值

$\text{lowbit}(x) = x \& (-x)$

$C_i = A_{(i-\text{lowbit}(i)+1)} + \dots + A_i$

# 修改与查询代码

```
int sum(int x) {  
    int s = 0;  
    while (x > 0) {  
        s += c[x]; x -= lowbit(x);  
    }  
    return s;  
}
```

```
void add(int x, int d) {  
    while (x <= n) { c[x] += d; x += lowbit(x); }  
}
```

# 乒乓比赛

$n$ 个人按次序排列，每个人乒乓球技能值为 $a_i$ ，互不相同  
每场比赛需要两名选手，一名裁判，裁判的位置和技能值都必须在中间  
问一共能组织多少场这样的比赛 ( $n \leq 20000$ )

分析：

枚举裁判 $i$ ，计算 $i$ 之前有多少比 $a_i$ 小， $i$ 之后又有多少比 $a_i$ 小  
以数值作为下标，开两个树状数组即可

# 推销员

去一处死胡同推销， $n$ 家用户，第 $i$ 家用户离入口距离为 $s_i$ ，推销积累疲劳值 $a_i$   
每走1距离也会积累疲劳值1，需要推销 $x$ （ $x$ 从1取到 $n$ ）家  
不走多余路，最多能积累多少的疲劳值( $n \leq 100000$ )

分析：

若采用 $O(n^2)$ 算法，每一次增加，要么在距离内、要么在距离外找  
满分要采用数据结构维护

树状数组（一个维护前缀，只记录疲劳值，一个维护后缀，疲劳值加距离）



# 数星星

$n$  ( $n \leq 100000$ ) 颗星星有各自坐标  $(x, y)$  ( $0 \leq x, y \leq 100000$ )

对于每颗星星，求两个坐标均不大于它的星星个数

分析：

这是一道二维数点问题

我们按横坐标把星星排序

则把纵坐标放入树状数组中即可

# 魔兽争霸

有 $n$ 个食尸鬼，初始生命值为 $a_i$ ，有三种操作：

第 $i$ 个食尸鬼被攻击，扣了 $d$ 的生命值

第 $i$ 个食尸鬼被死亡骑士发动死亡缠绕，恢复 $d$ 的生命值

询问生命值第 $k$ 多的食尸鬼有多少生命值

分析：

将食尸鬼的生命值当做下标，放入树状数组

二分答案求第 $k$ 大

# Apple Tree

有一棵 $n$ 个节点的苹果树，一开始每个节点上都结有一个苹果  
每次可以把一个点上的苹果摘下，或让一个没有苹果的节点长出一个苹果  
也可以询问以某节点为根的子树中有多少苹果（数据为10万数量级）

分析：

树形结构无法用树状数组

我们可以dfs遍历得出dfs序

树就成了序列，一棵子树就成了一个区间

树状数组维护即可

# 单点查询，区间修改

区间修改，指的是把序列中从l到r的数都加上d

我们对序列进行差分操作

令 $c_i = a_i - a_{i-1}$ ,  $c_1 = a_1$ , 维护b数列

这样区间修改就成了  $c_{r+1} -= d$ ,  $c_l += d$

单点查询就成了求 $c_1 + c_2 + \dots + c_i$

# 简单题

一个01序列，每次让一个区间内的数翻转，询问第 $l$ 个数是多少

分析：

我们依然采用差分，发现行不通

0与1的翻转让我们想起了异或

于是用异或运算来做差分

另一个方面理解，我们可以把翻转操作看成加1

最后只关心某个数的奇偶性

# 区间修改，区间查询

仍然使用差分的方法，考虑 $a_1..a_x$ 区间和的计算。

$c[1]$ 被累加了 $x$ 次， $c[2]$ 被累加了 $x-1$ 次， $\dots$ ， $c[x]$ 被累加了1次

$$\sum a[i] = \sum \{c[i] * (x - i + 1)\} = \sum \{c[i] * (x + 1) - c[i] * i\} = (x + 1) * \sum c[i] - \sum (c[i] * i)$$

再用树状数组维护一个数组 $c2[i] = c[i] * i$ ，即可算出结果

### 3、并查集

# 算法描述与实现

给定一些元素，并给出一些“某两个元素属于同一集合”的关系  
询问任意一个元素属于哪个集合

$f[i]$ , 表示 $i$ 所属集合的代表元

“并”  $x, y$ 属于同一集合，则 $u = \text{getfather}(x)$ ,  $f[u] = y$ ;

“查” `int getfaher(int x) { return f[x] == x ? x : f[x] = getfather(f[x]); }`

这里用到了路径压缩



# 亲戚

有 $n$ 个人，我们给出 $m$ 条形如“ $a$ 与 $b$ 是亲戚”的信息。  
再给出一些形如“ $c$ 与 $d$ 是否是亲戚”的询问。

分析：

每个人看成一个点或者一个元素

亲戚关系看成边，则这是一个很明显的并查集的问题

# 说谎岛

有M个问题，N个人来回答，每个人回答两个问题  
每个问题的答案为“是”或“否”，每个人说一句真话，说一句假话  
请问这N个问题的答案有多少种可能的情况？

分析：

从输入N，M来看，这很像一道图论题目

假设一个人说a是假，b是真，那么有两种情况

要么a假b假，要么a真b真，我们就给点(a,0),(b,0)连一条边,(a,1),(b,1)连边

其他情况同理，这就是并查集

如果(x,0)，(x,1)属于同一个集合，那么无解，否则解形如 $2^t$

# 奇偶

有一个由0和1组成的长度 $\leq 10^9$ 的序列，现在有n条信息  
每条信息的形式：a b even/odd，说明a到b位置共有偶数/奇数个1  
判断第一个不正确的信息的位置

分析：

前缀和 $S_i$ ，a,b even表示 $S(a-1)$ 与 $S_b$ 奇偶性相同，连 $(a-1,0)(b,0)$ ,  $(a-1,1)(b,1)$   
若出现 $(x,0)$ 与 $(x,1)$ 属于同一个集合，则矛盾

长度太大？ 离散化

每做一条全部判断一遍效率太低？ 二分答案

# 易爆物

有一些简单化合物，每种化合物由两种元素组成  
若 $k$ 种化合物恰好含有 $k$ 中元素，则有危险  
求问是否危险？

分析：

把每种元素看成一个点，则一个化合物就是一条边  
题目就转化为判断一张图是否有环  
这可以用并查集来判断

# 合作网络

$n$ 个结点，初始时每个结点的父节点都不存在，两种操作：

$I\ u\ v$ ：把 $u$ 的父亲设为 $v$ ，距离为 $|u-v| \bmod 1000$

$E\ u$ ：询问 $u$ 到根节点的距离。

分析：

根就是并查集的代表元

路径压缩时维护距离

```
if (f[x] != x) { int root = getfather(f[x]); d[x] += d[f[x]]; return f[x] = root };  
else return x;
```

# 代码等式

$1bad1 = acbe$ ,  $a, b, c, d, e$ 都是01串, 长度分别为4,2,4,4,2  
问有几种可能的解?

分析:

将一个长度为 $l$ 的变量分解为 $l$ 个长度为1的变量

这样我们就得到了 $N$ 个等式, 再次用 $(x, 0), (x, 1)$ 这样的方法

由于有数字介入, 还要加上0,1这两个点

# 团伙

有 $n$ 个人，任何两个认识的人不是朋友就是敌人

一个人朋友的朋友是他的朋友

一个人敌人的敌人是他的朋友

所有是朋友的人组成一个团伙

给出 $m$ 条朋友或敌人信息

问最多有几个团伙？

# 团伙

分析：

我朋友的敌人，我敌人的朋友，都不能确定是不是我的敌人！！！！

对于朋友信息，我们显然用并查集来维护。

一个人的所有敌人一定在同一个团伙，因此给每个 $i$ 记录他的敌人集合 $e[i]$

对于一条敌人信息，把 $i$ 和 $e[j]$ 合并， $j$ 和 $e[i]$ 合并



# 蜗牛

有 $n$ 只蜗牛，爬行速度 $1 \dots n$ ，一开始都在井外  
每次打开井盖，要么等待一只蜗牛爬上来并盖上井盖  
要么丢一只蜗牛进去并盖上井盖  
蜗牛爬出后不会再被丢进去  
盖上井盖后所有蜗牛都跌入井底  
求蜗牛爬出井盖的顺序

分析：

维护一个序列，支持插入和删除最大值

# 蜗牛

我们关注取最大值操作M，在最后加入 $n-m$ 次虚拟的求最大值操作  
记第 $i-1$ 次Max操作和第 $i$ 次Max操作之间的插入序列为 $S_i$

如果蜗牛 $n$ 在 $S_j$ 中，则 $M_j$ 的答案一定为 $n$ 。

删除 $M_j$ ,并把 $S_j$ 和 $S_{j+1}$ 合并，再查找蜗牛 $n-1$ 所在序列，依次类推  
用并查集来维护S序列。

# 可爱的猴子

树上挂着 $n$ 只可爱的猴子

猴子1的尾巴挂在树上，猴子的两只手都可以抓住一只猴子的尾巴

每一秒，都有一只猴子把某只手松开

求每只猴子的落地时间

分析：

和并查集问题恰好相反，所以我们倒过来做这个问题

就变成求每只猴子第一次与猴子1相连的时刻

除并查集外，还要一个链表，来遍历一个集合里的所有猴子

# 4、堆

# 堆的特点与操作

堆是一棵完全二叉树

以小根堆为例，每一个点的元素都小于子节点的元素

堆顶元素就是整个堆中的最小值

优先队列 `priority_queue<int> q;`

操作：

读取堆顶元素 `q.top();`

弹出最小值 `q.pop();`

插入一个新的值 `q.push(x);`

# k个最小和

有k个整数数组，各包含k个元素

每个数组取一个数相加，能得到 $k^k$ 个和，求这些和中最小的k个值

分析：

简化版，只有两个数组

$A_1+B_1 \leq A_1+B_2 \leq \dots$

$A_2+B_1 \leq A_2+B_2 \leq \dots$

$A_n+B_1 \leq A_n+B_2 \leq \dots$

n个元素放入堆中，记录和s与B的下标

# 积水

一块土地被划分成 $N*M$ 个正方形小块，每块地的高度为 $H(i,j)$   
请问它最多能积存多少降水？

分析：

看似无从下手，从特殊情况入手，如边界格子积水一定为0

从边界最低的格子 $x$ 入手，假设它的高度为 $h$ ，它相邻的格子是 $y$

如果 $y$ 的高度小于 $h$ ，则 $y$ 的水位为 $h$

对 $y$ 周围的格子“灌水”，直到它们被高度大于 $h$ 的格子包围起来

而被包围的这一圈是不能积水的，否则会转移到 $x$ 上而流出

于是不能积水的格子又围成了一圈，继续找最小的，用堆维护

# 赛车

$n$ 辆赛车在一条直线上，初始位置 $x_i$ （递增），速度为 $v_i$ ，都向右驶去  
不断有超车现象发生，输出超车的总数，以及按照时间排序前 $m$ 个超车时间

分析：

第一问是逆序对数目，可以用树状数组解决

第二问模拟事件，用堆维护

下一超车事件一定是某辆车追上了当前离它最近的车

每辆车 $i$ 和它前一辆车 $e[i]$ 的编号放在堆里，计算超车绝对时间排序

若 $i$ 车超过了 $j$ 车，则 $e[x] = i$ 的 $x$ 要改为 $e[x] = j$ ， $e[i]$ 更新为 $e[j]$ ， $e[j]$ 不用更新

如何确定 $x$ ？同时再记录一个 $f[i]=x$



# 可怜的奶牛

有 $n$  ( $n \leq 100000$ ) 头奶牛，每天产奶量最少的一头奶牛会被吃掉  
如果有多头奶牛产奶最少，当天可以幸免  
第 $i$ 头奶牛产奶有周期 $T_i$  ( $\leq 10$ )，每天产奶量不超过200  
问最后能剩下多少头奶牛？

分析：

把周期同为 $t$ 的奶牛合并起来，每天只需比较10类牛中最小产奶即可  
吃掉一头周期为 $t$ 的牛后，我们修改周期为 $t$ 的堆，删除最小值并维护

# 最轻巧的语言

字母表 $A_k$ 由 $k$ 个字母组成，每个字母都有一个权值。

一个单词的权等于字母权值之和

希望找 $n$ 个单词权值和最小，且任意一个单词不是其它单词的前缀

( $k \leq 26$ ,  $n \leq 10000$ , 权  $\leq 10000$ )

分析：

考虑一棵Trie树

一开始的单词为 $\{a, b, c, \dots\}$ ，每次取出权值最小的叶子来扩展，用堆维护一个小根堆还不够，需要再一个大根堆，且互相记录位置

# 黑匣子

一个序列初始为空， $k=0$ ，每次可以插入一个元素，也可以删除一个元素  
也可以 $k++$ 并询问第 $k$ 小元素

分析：

堆只能求第1小元素，现在要求第 $k$ 小，而且 $k$ 只增不减

我们用两个堆来维护，一个小根堆，一个大根堆

# 5、RMQ与LCA

# RMQ

求一个序列中  $A_L, \dots, A_R$  的最小值。序列不会被修改，询问有多个。

分析：

若是求和，我们可以用前缀和来维护，但最小值没有前缀最小值的方法

$d(i,j)$  表示从  $A_i$  开始，连续  $2^j$  个数的最小值

计算  $d$  数组：  $d[i][j] = \min( d[i][j-1], dp[i+(1 \ll (j-1))][j-1] );$

计算答案：  $\text{return } \min( d[l][k], d[r-(1 \ll k)+1][k] );$

# 频繁出现的数值

给出一个非降序排列的整数数组

对于一系列询问  $(i, j)$ ，回答  $a_i, a_{i+1}, \dots, a_j$  中出现次数最多的值所出现的次数

分析：

所有相等元素会聚在一起

我们把相等的一段用它的长度来记录

并记下每一个位置所在段的左端点和右端点

题目转换为求区间内的最大值，成为了一个RMQ问题

# GCD

一个长度为 $n$ 的整数序列，给出 $n$ 个询问区间

求指定区间的最大公约数

并求区间 $[1, n]$ 中有多少个区间的公约数等于求出来的这个公约数

分析：

求最大公约数，由于 $\gcd(a, \gcd(b, c)) = \gcd(\gcd(a, b), c)$ ，可以用同样方法

第二问，预处理出每一个 $a_i$ 开头，最大公约数会是多少，只有连续的几段

# LCA

给定一棵树，求两个点的最近公共祖先

$\text{dep}[u]$ 表示 $u$ 的深度

$f[u][k]$ ，表示 $u$ 向上走 $2^k$ 的祖先

$p[u][k]$ ，附带的统计信息

找 $u, v$ 的最近公共祖先时

先调到深度相同，再往上走



# 邦德

$n$ 个点， $m$ 条边，每条路都有危险系数

回答若干个询问， $s$ 到 $t$ 的路中所有边的最大危险系数最小

分析：

我们用并查集，发现 $s$ 到 $t$ 的路径全是最小生成树上的边

可以求最小生成树，然后用LCA的方法

也可以二分答案，用并查集来判断

# Network

给定一棵树，再给出m条非树边，去掉一条树边和一条非树边，使树断裂  
问共有几种方法？

分析：

每一条非树边(u,v)对应树上一条路径(u->lca(u,v)->v)，将这些边都标记一次  
被标记过0次的边，直接删去树就断了；标记过两次以上的边，删了无用  
被标记过一次的边，只有唯一一条非树边一起删去才会断  
如何标记呢？ 每次 $dp[u]++$ ,  $dp[v]++$ ,  $dp[lca(u,v)] -= 2$ ;  
最后统计子树的dp值之和。

# The Merchant

给出一棵节点有值的树，给出Q个询问(a,b)，问从a到b的最大盈利。

盈利指的是，沿着道路走，在某一点买入，再在另一点卖出

分析：

先考虑序列上的情况，采用分治

于是我们用LCA的方法，记录最大值，最小值，（向上或向下走）最大盈利

接下来求(a,b)两点LCA，求的时候维护即可

# Checkers

数轴上有三个点  $(a,b,c)$ ，每次可以一个点为中心，将另一个较近的点对称。  
问初始状态 $(a,b,c)$ 能否变为目标状态  $(x,y,z)$ ，求最少步数

分析：

把状态 $(x,y,z)$ 看作点， $y$ 往两边跳对应两个儿子节点，另一种跳法对应父节点  
形成森林的结构，两个状态最少步数，相当于求LCA  
在求深度以及向上走特定的步数的时候，我们用辗转相除法

## 6、线段树

# 线段树操作

思想：二分

所用数据结构：二叉树

单点修改

区间查询

区间修改（加上某一个值）

区间修改（全部改为某一个值）

采用打标记，释放标记的方法

# 集合

有三种类型的操作，

1."add x"表示往集合里添加数x。

2.“del x”表示将集合中数x删除。

3.“sum”求出从小到大排列的集合中下标模5为3的数的和。

集合中的数都是唯一的。

分析：

把数作为下标，线段树中记录区间内模五为0...4的数字和，以及有多少个数  
合并时，左子区间直接算，右子区间作相应位移

# LIS

一个序列，可以区间加值，询问某一区间内连续最长上升子序列

分析：

对于每个区间，记录最长连续LIS

为了左右区间合并，要记录前缀最长及后缀最长LIS，还要记录端点值



# 上帝造题的七分钟

数列中的数 ( $\leq 10^{12}$ ) 支持区间开平方 (下取整) 操作, 以及区间求和操作

分析:

序列中的数开6次平方就变成了1

所以可以记录每个区间是否全是1, 不是的话, 往下做

# 括号序列

给一个括号序列，可以区间改值，可以区间取反，询问区间是否为合法序列

分析：

将左右括号分别看作0,1,则序列的和为0，最小前缀和  $\geq 0$

为此，维护序列和，最小前缀和，由于有取反，还要维护最大前缀和  
两个标记setv[o]，rev[o]先放下setv标志，再放下rev标志

# 等差数列

序列的长度为250000，有四种操作

操作A，从st到ed这段区间内的数，分别加上 $1, 2, \dots, st-ed+1$ 。

操作B，从st到ed这段区间内的数，分别加上， $st-ed+1, st-ed, \dots, 1$ 。

操作C，将st到ed这段区间内的数赋值成x。

操作S，查询st到ed的这段区间内的数的总和。

分析：

操作C可用setv[o]操作符来进行

操作A，B都是加上一个等差数列，记录一段内的首项，末项即可

# Sequence Operation

有N个为0或为1的数，有M个操作，操作有5种类型。

- (1) "0 a b"，表示将区间[a,b]范围内的数全部置0。
- (2) "1 a b"，表示将区间[a,b]内的数全部置1。
- (3) "2 a b"，表示将区间[a,b]内的数0变成1，1变成0。
- (4) "3 a b"，表示查询[a,b]范围内1的数。
- (5) "4 a b"，表示查询[a,b]范围内最长的连续的1。

分析：

分治思想，记录每一线段内最长的0,1序列，最长的前缀、后缀0,1序列

# Little Elephant and Array

给定一个序列，若干个询问，区间 $[l,r]$ 内有多少值为 $v$ 的数出现次数也为 $v$

分析：

离线处理所有询问，按终点从小到大排序

$\text{cnt}[\text{value}]$ 表示 $\text{value}$ 出现的次数， $\text{pos}[\text{value}]$ 表示 $\text{value}$ 的各个位置

在线段树上单点标记上1,-1等值，区间统计即可

# 扫描线

给出N个矩形，求它们的面积并

分析：

按左下角的横坐标排序，一条扫描线经过

将y坐标作为下标建立线段树,区间加减，统计非0的长度

# Necklace

有T组数据，每组数据有N个数，有Q个查询

每个查询要求区间 $[l, r]$ 之间数的和，但是出现多次的某个值只能算出现一次

分析：

离线处理，将所有询问按终点坐标从小到大排序

$\text{cnt}[\text{value}]$ 表示value出现的次数， $\text{pos}[\text{value}]$ 表示value的各个位置

扫描时，最后一个位置的值加入线段树中，其他的位置都置为0

# 借教室

每一天的教室数为 $a[i]$ ，第 $j$ 个请求要在第 $l_j$ 天至第 $r_j$ 天借教室，最多能满足前几个请求？

分析：

很容易想到的方法，线段树区间修改，区间统计最小值

但是线段树常数大，容易超时，且线段树编写较为麻烦

采用上题方法，每一天的请求打标记，起点+1，终点后一位-1

二分答案，打标记，从左往右扫描



## 7、set与map

# 基本操作

```
set<int> s;
```

```
s.clear(); s.begin(); s.end(); s.size();
```

```
s.insert(P); s.erase(P);
```

```
set<int>::iterator it; it = s.lower_bound(x); 返回第一个大于等于x的迭代器  
*it读取改值
```

```
map<int, int> a;
```

```
a.clear()
```

```
a.count(x);
```

```
a[x] = v;
```

# 最短路

n个节点，m条边，求最短路

每个结点没有编号，只有字符串名

分析：

每个字符串名和一个数字一一对应起来

map[string] = int，用map很方便

# 2的幂

一个正整数序列被称为好的，当且仅当  
对于任意 $a_i$ ，能找到一个 $a_j$  ( $i \neq j$ )且 $a_i + a_j$ 为2的幂  
给定一个序列，问至少删去多少个数，才能使它变好

分析：

从左到右枚举，如果一个数找不到对子，就把它删去  
如何判断有无对子？map存储

# 数星星（加强版）

一个星星的坐标是三维的，求每个星星下有几颗星星

分析：

二维时候用树状数组，三维时候用二维树状数组

空间不够？我们只会用到某些值，开map

对于 $(x,y)$ ，采用 $x*M+y$ 与之对应， $M$ 为一大整数

# 8、平衡树

# Treap

二叉排序树，平衡树

手动实现set，并能拓展其他功能

```
struct Node
{
    Node* ch[2];
    int r, v;           //r为优先级，v为值（优先级随机给定，下小上大）
    bool operator < () ..... （比较优先级大小）
    inc cmp(int x) const ..... (比较x与值v的大小)
}
```

# Treap

```
void rotate(Node* &o, int d) {    // d = 0表示左旋, d = 1表示右旋
    Node* k = o->ch[d^1]; o->ch[d^1] = k->ch[d]; k->ch[d] = o; o = k;
}
```

```
void insert(Node* &o, int x) {
    if (o == NULL) {
        o = new Node(); o->ch[0] = o->ch[1] = NULL; o->v = x; o->r = rand(); }
    else {
        int d = o->cmp(x);
        insert(o->ch[d], x); if (o->ch[d] > o) rotate(o, d^1); }
}
```



# Treap

```
int find(Node* o, int x) {  
    while (o != NULL) {  
        int d = o->cmp(x);  
        if (d == -1) return 1;    //找到了x值  
        else o = o->ch[d];  
    }  
    return 0;    //没有找到x值  
}
```

# Treap

名次树：每个结点有一个附加域，表示子树的大小  
可以找出第k小的元素，也可以求出x的名次  
需要一个maintain函数来维护

```
size = ch[0]->size + ch[1]->size + 1;
```

在rotate操作时，需要在o=k之前加上  
`o->maintain(); k->maintain();`

# Splay

如果对一些序列，我们要能进行合并或者分裂，甚至反转

Treap不能提供这样的功能，我们需要用Splay来解决

新的功能：快速地分裂与合并

伸展操作：把一个指定结点自底向上转到根结点

基础操作：旋转

结点定义方式同Treap，依然是Node指针

# Splay

```
void splay(Node* &o, int k) {  
    int d = o->cmp(k);  
    if (d == 1) k -= o->ch[0]->s - 1;  
    if (d != -1) {  
        Node* p = o->ch[d]; int d2 = p->cmp(k);  
        int k2 = (d2 == 0 ? k : k - p->ch[0]->s - 1);  
        if (d2 != -1) {  
            splay(p->ch[d2], k2);  
            if (d == d2) rotate(o, d^1); else rotate(o->ch[d], d); }  
        rotate(o, d^1); } }  
}
```

# Splay

```
Node* merge(Node* left, Node* right) {  
    splay(left, left->size); left->ch[1] = right;  
    left->maintain(); return left;  
}
```

```
void split(Node* o, int k, Node* &left, Node* &right) {  
    splay(o, k);  
    left = o; right = o->ch[1];  
    o->ch[1] = NULL; left->maintain();  
}
```

# Play with Chain

给定一个序列，有两个操作

一是把一段区间翻转，二是把一段区间截下来放到另外一个位置  
求最后的序列

分析：

第二个操作就是splay所擅长的

第一个操作需要用到标记flip

# 维护数列

请写一个程序，要求维护一个数列，支持以下 6 种操作：（请注意，格式栏中的下划线‘\_’表示实际输入文件中的空格）

| 操作编号       | 输入文件中的格式                    | 说明                                                                                |
|------------|-----------------------------|-----------------------------------------------------------------------------------|
| 1. 插入      | INSERT_posi_tot_c1_c2..._cn | 在当前数列的第 $posi$ 个数字后插入 $tot$ 个数字： $c_1, c_2, \dots, c_{tot}$ ；若在数列首插入，则 $posi$ 为 0 |
| 2. 删除      | DELETE_posi_tot             | 从当前数列的第 $posi$ 个数字开始连续删除 $tot$ 个数字                                                |
| 3. 修改      | MAKE-SAME_posi_tot_c        | 将当前数列的第 $posi$ 个数字开始的连续 $tot$ 个数字统一修改为 $c$                                        |
| 4. 翻转      | REVERSE_posi_tot            | 取出从当前数列的第 $posi$ 个数字开始的 $tot$ 个数字，翻转后放入原来的位置                                      |
| 5. 求和      | GET-SUM_posi_tot            | 计算从当前数列开始的第 $posi$ 个数字开始的 $tot$ 个数字的和并输出                                          |
| 6. 求和最大的子列 | MAX-SUM                     | 求出当前数列中和最大的一段子列，并输出最大和                                                            |



给定一个图，有一些无向边，现在有三种操作

1、改变点权， 2、删掉一条边， 3、查询结点 $x$ 所在连通块第 $k$ 大的点权。

分析：

删边不好办，连边比较容易，离线所有操作，从后往前算

合并时启发式合并，小的往大的加

点的权值用领接表存储



# 列队

$n$ 行 $m$ 列方阵，同学依次编号

每次有一个同学离队，然后向左看齐，向前看齐

问每次离队同学的编号

分析：

我们初始状态每行有一个节点，节点有一个区间从第1人到第 $m-1$ 人

第 $n+1$ 个splay维护最后一列，一共 $n$ 个节点。

这样每个节点都是一个连续的区间，操作时把区间拆开就行